

Week 6 · 개념 워크북: 코드와 state와 실물이 어긋날 때 무엇을 어느 쪽에 맞출 것인가

📌 참고

이 문서는 개념 파트입니다. 명령을 직접 입력하는 실습은 같은 `lecture/` 폴더의 **실습워크북** 을 따라가세요. **[개념]** 은 개념 설명, **[함정]** 은 미리 알고 피해 가는 것입니다.

지금까지 다섯 주 동안 순서가 언제나 같았습니다. 코드를 먼저 쓰고, `apply` 하면 AWS 에 생기고, state 에 기록이 남았습니다. 이번 주에는 그 순서가 두 번 깨집니다. 한 번은 콘솔이 나중에 개입하고, 한 번은 콘솔이 먼저 만들어 놓습니다.

이 문서를 덮을 때 세 가지를 말할 수 있어야 합니다. 코드와 state와 실물이 각각 무엇인지, 셋이 어긋났을 때 `plan` 이 그것을 어떻게 보여주는지, 그리고 코드 밖에서 태어난 리소스를 Terraform 안으로 들여오는 절차가 무엇을 해주고 무엇을 안 해주는지입니다.

! 꼭 기억

week1·2·3·4·5에서 이미 설명한 것은 다시 풀지 않고 위치만 가리킵니다. `plan` · `apply` · `destroy` 의 흐름은 week1 개념워크북에, state 파일과 리소스 주소는 week2 개념워크북 9 · 12번에, 원격 backend 와 잠금은 week3 개념워크북 Part 0에, `variable` · `for_each` · `sensitive` 는 week4 개념워크북에, 모듈과 `module.` 접두사와 `moved` 는 week5 개념워크북에 있습니다.

오늘은 week5 개념워크북 17번이 Week6로 넘긴 표를 그대로 회수합니다.

week5에서 넘긴 것	오늘 어디서 받는가
리소스 주소가 <code>module.network.aws_vpc.this</code> 가 되었습니다. 콘솔에서 직접 만든 리소스는 이 주소를 갖고 있지 않습니다	Part 4의 10번
<code>moved</code> 는 AWS 에 요청을 보내지 않고 state 의 주소만 고칩니다. state 만 따로 고칠 수 있다는 것은 state 와 실물이 어긋날 수 있다는 뜻입니다	Part 1의 1 · 2번
<code>plan</code> 이 <code>9 to add</code> 인지를 안전 게이트로 썼습니다. 누가 콘솔에서 값을 바꾸면 <code>~</code> 가 뜹니다	Part 2의 4 · 5번
모듈은 계약만 보이고 안은 보이지 않습니다. 모듈 안 리소스를 import 하려면 <code>module.network.</code> 까지 붙인 주소를 적어야 합니다	Part 5의 15번
<code>state list</code> 가 비었어도 계정 쪽은 따로 확인했습니다	Part 1의 3번

오늘 다룰 것

📌 참고

세션은 60분이고 그중 45분이 실습입니다. 그래서 이 문서는 대부분 연습용입니다. 라이브에서 다루는 것은 **Part 1 · 2 · 3** 과 **Part 5의 13 · 14번** 인데, 개념만 따로 떼어 설명하는 시간은 따로 두지 않습니다. 실습워크북 Block A 워크스루(18분) 안에서 직접 실행하면서 함께 설명합니다. 나머지는 읽고 오세요. 읽지 않으면 실습 중에 화면에 뜬 ~ 와 + 가 무슨 뜻인지 모른 채 명령만 따라 입력하게 됩니다.

Part	섹션	무엇을 다루나	언제
Part 0	용어 사전	오늘 처음 나오는 말 13개	예습
Part 1	1 · 2 · 3	코드 · state · 실물 세 곳	예습 · Block A 에서
Part 2	4 · 5 · 6	plan 을 읽는 법	예습 · Block A 에서
Part 3	7 · 8 · 9	어긋남을 어느 쪽으로 맞출 것인가	Block A 에서
Part 4	10 · 11 · 12	코드 밖에서 태어난 리소스	예습 · Block B 에서
Part 5	13 · 14 · 15	고전 import 의 한계	Block B 에서
Part 6	16 · 17	비용과 정리	Block C 에서
Part 7	18	다음 주로 넘기는 문제	예습

오늘 만드는 것

S3 버킷 두 개뿐입니다. 하나는 코드가 먼저 만들고 나중에 콘솔이 손을 땁니다. 다른 하나는 콘솔이 먼저 만들고 나중에 코드가 따라잡니다. 이번 주에는 EC2 를 만들지 않고 모듈도 쓰지 않습니다.

버킷은 둘인데 Terraform 리소스로 세면 넷이 됩니다. 왜 그렇게 되는지가 Part 5의 14번입니다.

📌 참고

이 문서에는 명령과 화면을 신지 않았습니. 무엇을 어떤 순서로 치고 무엇이 나와야 하는지는 실습워크북의 "오늘 나와야 하는 숫자" 표와 각 스텝에 있습니다. 이 문서는 그 화면이 왜 그렇게 나오는지만 다룹니다.

실습 중에는 실습워크북만 펴고 있으면 됩니다. 이 문서는 세션 전에 읽는 것입니다.

Part 0. 용어 사전

오늘 처음 나오는 단어를 실습 전에 다 풀어둡니다. week1~week5에서 정의한 용어는 넣지 않았습니. 비유는 이 표 안에서만 쓰고, 본문에서는 왼쪽 열의 실제 이름으로 부릅니다.

용어	뜻	비유
실물 (real infrastructure)	AWS 계정 안에 실제로 존재하는 것. 콘솔에서 눈으로 보는 그것입니다	-
드리프트 (drift)	코드 · state · 실물 중 어느 둘 이상이 어긋난 상태. "표류"라는 뜻의 영어 단어입니다	배가 닻에서 밀려난 것
refresh	Terraform 이 AWS 에 "지금 실제로 어떻게 되어 있나요"를 물어보는 동작. <code>plan</code> 과 <code>apply</code> 가 시작할 때 자동으로 합니다	-
<code>-refresh-only</code>	<code>apply</code> 에 붙이는 옵션. 실물은 수정하지 않고 state 만 실물에 맞춰 고칩니다	-
<code>-refresh=false</code>	<code>plan</code> 이나 <code>apply</code> 에 붙이는 옵션. AWS 에 묻지 않고 state 값만 믿고 계산합니다	-
진실의 원천 (source of truth)	셋이 어긋났을 때 어느 쪽을 정답으로 볼 것인가. Terraform 을 쓰는 동안에는 코드입니다	-
<code>import</code>	이미 존재하는 실물을 state 에 적어 넣어서 Terraform 의 관리 대상으로 만드는 명령	전학 온 학생을 출석부에 올리는 것
import ID	import 할 때 "AWS 의 어느 것인지" 가리키는 값. 리소스 종류마다 형식이 다릅니다	-
고전 import (classic import)	<code>terraform import</code> 명령으로 하나씩 편입하는 방식. HCL 은 사람이 씁니다. 오늘 하는 것	-
<code>import</code> 블록	Terraform 1.5부터 생긴 방식. 코드에 적어두면 <code>apply</code> 가 편입까지 합니다. 7주차	-
부속 리소스	하나의 실물처럼 보이지만 Terraform 에서는 별도 리소스로 나뉜 것. S3 의 버전 관리 · 퍼블릭 접근 차단이 그렇습니다	-
<code>state show</code>	state 에 적힌 값을 그대로 보여주는 명령. AWS 에 묻지 않습니다	출석부만 펴서 보기
관리되지 않는 리소스 (unmanaged)	AWS 에는 있는데 어느 state 에도 없는 것. Terraform 은 존재 자체를 모릅니다	출석부에 없는 학생

Part 1. 코드 · state · 실물 세 곳

[개념] 1. 같은 값이 세 곳에 적혀 있습니다

지금까지 우리가 다룬 것은 두 가지처럼 보였습니다. 내가 쓰는 `.tf` 파일과 AWS 에 생기는 것입니다. 그런데 그 사이에 state 가 있었습니다. week3에서 그것을 S3 버킷으로 이동했고, week5에서 `module.` 접두사가 붙은 주소를 거기서 확인했습니다.

오늘 만드는 버킷의 `Env` 태그를 예로 들면, 같은 값이 세 곳에 적혀 있습니다.

코드 main.tf	state S3 의 week06/app/ terraform.tfstate	실물 AWS 의 버킷
Env = "study"	"Env": "study"	Env : study
사람이 씁니다 git 에 올라갑니다	apply 가 씁니다 S3 에 올라갑니다	apply 가 만듭니다 AWS 계정 안에 있습니다

셋이 같으면 `plan` 이 `No changes` 라고 말합니다. 지난 다섯 주 동안 대부분 그랬습니다. 셋을 어긋나게 하는 길은 둘뿐입니다. 코드를 고치거나(우리가 지금까지 한 일), 콘솔에서 실물을 고치는 것입니다(오늘 할 일).

week5의 `moved` 블록을 떠올려 보세요. `moved` 는 AWS 에 요청을 보내지 않고 state 의 주소만 변경했습니다. **state 만 따로 변경할 수 있다는 것은, state 와 실물이 서로 다를 수 있다는 뜻입니다.** 그 가능성이 오늘의 주제입니다.

[개념] 2. Refresh: `plan` 이 매번 자동으로 수행하던 작업

`terraform plan` 을 치면 Terraform 이 세 가지를 합니다.

순서	하는 일	어디를 봅니까
1	state 를 읽습니다	S3 의 <code>week06/app/terraform.tfstate</code>
2	refresh. state 에 적힌 리소스가 지금 실제로 어떤 상태인지 AWS 에 물어봅니다	AWS API
3	그 결과와 코드를 비교해서 무엇을 바꿔야 하는지 계산합니다	코드

2번이 오늘 처음 이름을 붙이는 동작입니다. 지금까지도 매번 수행되었습니다. `plan` 출력 맨 위에 나오던 `aws_s3_bucket.drift_demo: Refreshing state...` 줄이 그것입니다. 그 줄을 지금까지는 신경 쓰지 않고 넘겼을 텐데, 오늘은 그 줄이 무엇을 하는지가 실습의 절반입니다.

여기서 헷갈리기 쉬운 것이 하나 있습니다. **Refresh 는 화면에 보여줄 값을 계산하기 위해 메모리에서만 일어나고, state 파일 자체를 변경하지는 않습니다.** 그래서 콘솔에서 태그를 바꾼 뒤 `terraform plan` 을 보면 바뀐 것이 diff 로 보이는데, 곧바로 `terraform state show` 를 열어보면 state 에는 아직 이전 값이 남아 있습니다. `state show` 는 AWS 에 묻지 않고 state 파일에 적힌 것을 그대로 읽어주기 때문입니다.

이 두 명령의 차이가 "state 는 실물의 복사본이고 자동으로 갱신되지 않는다"를 보여주는 가장 짧은 방법입니다. 실습워크북 A-4에서 이 순서로 확인합니다.

[개념] 3. 드리프트라는 말

코드 · state · 실물 셋 중 어느 둘 이상이 어긋난 상태를 드리프트라고 부릅니다. 원인은 다양합니다.

어떻게 생기는가	예
사람이 콘솔에서 직접 수정합니다	장애 대응 중에 급하게 시큐리티 그룹 규칙을 열었습니다
AWS 가 스스로 바꿉니다	인스턴스가 있던 하드웨어가 고장 나서 교체되었습니다
다른 도구가 바꿉니다	오토스케일링이 대수를 늘렸습니다
사람이 콘솔에서 지웠습니다	비용을 줄이려고 안 쓰는 것처럼 보이는 리소스를 삭제했습니다

오늘 실습에서 만드는 것은 첫 번째 경우입니다. 나머지도 결과는 같습니다. `plan` 에 변경 사항이 표시됩니다.

week5 실습워크북 C-2에서 `terraform state list` 가 비었는지와 AWS 계정 쪽이 비었는지를 따로 확인했습니다. 그때는 이유를 "state 가 비었다는 것이 계정이 비었다는 뜻은 아니기 때문"이라고만 적었습니다. 오늘 그 상태에 이름이 붙습니다. state 에 없는데 계정에 남아 있는 것이 관리되지 않는 리소스이고, 그것을 Terraform 안으로 들여오는 절차가 Part 4의 `import` 입니다.

Part 2. `plan` 을 읽는 법

지난 다섯 주 동안 `plan` 의 마지막 줄 숫자만 봤습니다. 오늘은 그 위의 본문을 읽습니다.

[개념] 4. 기호 네 개

`plan` 출력에서 각 줄 앞에 붙는 기호가 무엇을 하겠다는 뜻인지 정해져 있습니다.

기호	이름	뜻	마지막 줄의 어느 숫자에 들어가나
<code>+</code>	create	새로 만듭니다	<code>to add</code>
<code>-</code>	destroy	지웁니다	<code>to destroy</code>
<code>~</code>	update in-place	지우지 않고 그 자리에서 고칩니다	<code>to change</code>
<code>-/+</code>	replace	지우고 다시 만듭니다	<code>to add</code> 와 <code>to destroy</code> 양쪽에

`~` 와 `-/+` 의 차이가 중요합니다. 속성 중에는 만든 뒤에 바꿀 수 있는 것과 없는 것이 있습니다. 태그는 언제든지 바꿀 수 있으니 `~` 로 처리됩니다. EC2 의 `subnet_id` 처럼 만들 때만 정할 수 있는 속성은 바꿀 방법이 없어서 `-/+` 가 되고, 그러면 인스턴스가 통째로 새로 만들어집니다. week5 개념워크북 9번에서 서버넷 맵의 key 를 지우면 EC2 가 교체된다고 한 것이 이 경우였습니다.

`-/+` 가 보이면 어느 속성이 교체를 일으켰는지도 함께 나옵니다. 그 속성 줄 끝에 `# forces replacement` 라는 주석이 붙습니다. 교체가 뜻밖이라면 그 주석을 찾아서 어느 값 때문인지부터 보세요.

오늘 실습에서는 `-/+` 가 나오지 않습니다. 태그만 바꾸기 때문입니다. 그래도 읽는 법은 알아두세요. 심화 과제에서 만날 수 있습니다.

🚫 절대 금지

`plan` 에서 `-` 나 `-/+` 를 봤는데 그것을 의도하지 않았다면 그 자리에서 멈추세요. `apply` 를 치는 순간 실물이 지워집니다. 지난 다섯 주 동안 마지막 줄 숫자를 확인하라고 계속 말한 이유가 이것입니다.

[개념] 5. 드리프트가 있을 때 `plan` 이 말하는 것

콘솔에서 태그 하나를 바꾸고 `terraform plan` 을 치면 화면이 두 덩어리로 나옵니다. 이 구조를 모르면 같은 태그가 화면에 두 번 나오는 것을 보고 헷갈립니다.

첫째 덩어리는 refresh 결과입니다. `Note: Objects have changed outside of Terraform` 이라는 제목이 붙고, Terraform 이 AWS 에 물어봤더니 state 와 다르더라는 보고가 이어집니다. 리소스 이름 옆에 `has changed` 가 붙습니다. 여기에 나오는 `~` 는 Terraform 이 하겠다는 일이 아니라 이미 벌어진 일입니다. 태그 값이 `study` 에서 `HACKED` 로 바뀌었다고 적힙니다.

둘째 덩어리가 실제 계획입니다. `Terraform will perform the following actions:` 아래에 나오고, 리소스 이름 옆에 `will be updated in-place` 가 붙습니다. 그리고 화살표 방향이 반대입니다. `HACKED` 에서 `study` 로 되돌리겠다고 적힙니다.

앞은 콘솔에서 벌어진 일이고, 뒤는 Terraform 이 그것을 되돌리겠다는 계획입니다. Terraform 은 코드를 정답으로 봅니다. 코드에 `Env = "study"` 라고 적혀 있으니 실물을 거기에 맞추려 합니다.

구분하는 방법은 간단합니다. 위쪽에 `has changed` (완료형), 아래쪽에 `will be updated` (미래형)입니다.

실제 화면은 실습워크북 A-5에서 두 덩어리를 나란히 놓고 봅니다.

📌 참고

`tags` 옆에 `tags_all` 이라는 것이 함께 나올 수 있습니다. `tags` 는 이 리소스에 직접 적은 태그이고 `tags_all` 은 provider 의 `default_tags` 까지 합친 결과입니다. 오늘 코드에는 `default_tags` 가 없어서 둘의 값이 같습니다. week4 개념워크북에서 `default_tags` 를 다뤘습니다.

[함정] 6. 콘솔에서 바뀌도 되는 것과 바꾸면 안 되는 것

드리프트를 만들어 보는 실습이라고 해서 아무 값이나 바꾸면 안 됩니다. 기준은 두 가지입니다.

첫째, 돈이 나가는 속성은 건드리지 않습니다. 인스턴스 타입을 콘솔에서 `t3.micro` 에서 `m5.large` 로 바꾸면 그 순간부터 요금이 올라갑니다. `plan` 으로 감지하고 `apply` 로 되돌리기까지 몇 분이 걸리고, 그동안 큰 인스턴스가 켜져 있습니다. 오늘 실습에서 태그를 고르는 이유가 이것입니다. 태그는 몇 개를 붙이든 요금이 붙지 않습니다.

둘째, 되돌릴 수 없는 것은 건드리지 않습니다. 콘솔에서 리소스를 삭제하면 `plan` 에 `+` 로 잡히고 `apply` 가 다시 만들어 주기는 합니다. 그런데 새로 만든 것은 ID 가 다르고, 그 리소스에 붙어 있던 데이터는 돌아오지 않습니다.

이번 주 실습에서 콘솔로 하는 일은 딱 둘입니다. drift 버킷의 `Env` 태그를 고치는 것과, manual 버킷을 새로 만드는 것입니다. 그 밖의 것은 손대지 마세요.

Part 3. 어긋남을 어느 쪽으로 맞출 것인가

셋이 어긋난 것을 알았습니다. 그다음에 할 수 있는 일이 두 가지입니다.

[개념] 7. `apply` : 실물을 코드에 맞춥니다

별도 옵션 없이 `terraform apply` 를 하면 계획대로 실행합니다. 태그가 `HACKED` 였던 실물이 `study` 로 돌아오고, state 에도 `study` 가 적힙니다. 셋이 다시 같아집니다.

이것이 기본 선택입니다. 코드가 진실의 원천이고, 콘솔에서 손댄 것은 사고이므로 되돌린다는 판단입니다.

[개념] 8. `apply -refresh-only` : state 를 실물에 맞춥니다

반대 방향도 있습니다. 콘솔에서 변경한 것이 옳고 코드가 낡은 경우입니다. `apply` 에 `-refresh-only` 옵션을 붙입니다.

이 명령은 AWS 에 아무 변경 요청도 보내지 않습니다. 실물을 그대로 두고 state 만 실물에 맞춰 업데이트합니다. 실행하면 승인을 묻는데, 문구가 평소와 다릅니다. 인프라를 변경하지 않고 state 만 업데이트하겠다는 안내가 붙습니다.

주의할 것이 하나 있습니다. `-refresh-only` 로 state 를 업데이트해도 코드는 그대로 `Env = "study"` 입니다. 그래서 곧바로 `terraform plan` 을 다시 실행하면 여전히 고칠 것이 남았다고 나옵니다. state 가 실물과 같아졌을 뿐, 코드와는 여전히 다르기 때문입니다.

정리하면 `-refresh-only` 는 어긋남을 없애는 명령이 아닙니다. 셋 중 둘(state 와 실물)을 먼저 맞춰서, 남은 차이가 코드와 실물 사이의 것임을 분명하게 만드는 명령입니다. 콘솔 변경을 받아들이기로 했다면 그다음에 코드를 직접 수정해야 셋이 같아집니다.

[함정] 9. 둘 중 무엇을 쓸 것인가

상황	무엇을 하는가	결과
콘솔 변경이 사고입니다. 코드가 맞습니다	<code>terraform apply</code>	실물이 코드로 되돌아옵니다
콘솔 변경이 의도된 것입니다. 코드가 낡았습니다	<code>terraform apply -refresh-only</code> 로 state 를 맞춘 뒤, 코드를 직접 수정합니다	셋이 같아집니다
무엇이 맞는지 아직 모릅니다	<code>terraform plan</code> 만 반복합니다	아무것도 바뀌지 않습니다

가장 흔한 실수는 두 번째 줄에서 `-refresh-only` 만 하고 끝내는 것입니다. state 와 실물은 같아졌지만 코드가 낡은 채로 남습니다. 다음 사람이 `apply` 를 하는 순간 실물이 코드 쪽으로 되돌아갑니다.

⚠ 주의

`plan` 과 `apply` 에 `-refresh=false` 를 붙이면 AWS 에 묻지 않고 state 만 믿고 계산합니다. 리소스가 많아 refresh 가 느릴 때 쓰는 옵션입니다. 그런데 이것을 붙이면 드리프트가 화면에 아예 안 나옵니다. 실습에서는 쓰지 마세요.

Part 4. 코드 밖에서 태어난 리소스

여기부터 방향이 반대입니다. 지금까지는 코드가 먼저였습니다. 이제 실물이 먼저입니다.

[개념] 10. state 에 없는 것은 Terraform 에게 없는 것과 같습니다

콘솔에서 버킷을 하나 만들었다고 해봅시다. AWS 에는 분명히 있습니다. 그런데 Terraform 의 state 에는 없습니다. `terraform plan` 을 아무리 쳐도 그 버킷은 화면에 나오지 않습니다. Terraform 은 state 에 적힌 것만 확인하기 때문입니다.

이런 리소스를 관리되지 않는 리소스라고 부릅니다. 회사에서 이것이 왜 문제가 되는지는 실습보다 큰 이야기입니다. 아무도 지우지 않는 리소스가 요금을 계속 먹고, 보안 설정이 코드 리뷰를 거치지 않고 들어가고, 담당자가 퇴사하면 왜 만들었는지 아는 사람이 없어집니다.

그래서 코드에 블록을 적어 관리 대상으로 만들려고 합니다. 여기서 초심자가 가장 많이 하는 실수가 나옵니다. 블록만 적고 `apply` 를 치는 것입니다.

```
resource "aws_s3_bucket" "manual" {
  bucket = "boaz26-w6-kdh1834-manual"
}
```

이 상태에서 `plan` 을 하면 이 리소스를 새로 만들겠다고 나옵니다. Terraform 은 state 에 이 리소스가 없으니 **없는 것으로** 판단합니다. AWS 에 그 이름의 버킷이 이미 있다는 것을 모릅니다. 그대로 `apply` 하면 이름이 겹쳐서 실패하거나, 리소스 종류에 따라서는 같은 것이 하나 더 만들어집니다.

필요한 것은 "만들어라"가 아니라 "이미 있는 저것이 이 코드가 참조하는 것이다"라고 알려주는 절차입니다. 그것이 `import` 입니다.

[개념] 11. terraform import 가 수행하는 작업과 수행하지 않는 작업

명령은 `terraform import` 뒤에 값을 둘 받습니다. 앞은 코드에 이미 적혀 있어야 하는 리소스 주소이고, 뒤는 AWS 에서 그것을 찾을 수 있는 import ID 입니다.

성공하면 "이 리소스들은 이제 당신의 Terraform state 에 있고 앞으로 Terraform 이 관리한다"는 안내가 나옵니다. 이 문장을 눈여겨보세요. 그 순간부터 그 버킷은 Terraform 의 관리 대상이고, `destroy` 의 대상이기도 합니다. 17번에서 다시 봅니다.

코드에 블록이 없으면 명령 자체가 거부됩니다. 그런데 거부하면서 어떻게 적으라는 예시까지 알려줍니다. 그 예시가 이번 주의 요점을 그대로 담고 있습니다. 리소스 블록의 형식만 보여주고 그 안은 `# (resource arguments)` 라는 주석 한 줄로 비워둡니다.

Terraform 이 채우는 것은 state 뿐이고, 괄호 안을 채우는 것은 사람입니다. 실제 오류 화면은 실습워크북 B-2에서 봅니다.

정리하면 이렇습니다.

import 가 수행하는 일	import 가 수행하지 않는 일
AWS 에서 그 리소스의 현재 상태를 읽어 state 에 기록합니다	<code>.tf</code> 파일에 속성을 작성해주지 않습니다
그 주소를 Terraform 관리 대상으로 만듭니다	실물을 전혀 변경하지 않습니다
이후 <code>plan</code> 이 그 리소스를 비교 대상에 포함시킵니다	부속 리소스까지 함께 편입해주지 않습니다

[개념] 12. import ID 는 리소스마다 다릅니다

오른쪽에 무엇을 적을지는 리소스 종류마다 정해져 있습니다. 외울 것이 아니라 provider 문서에서 찾는 것입니다. 각 리소스 문서의 맨 아래에 Import 절이 있습니다.

리소스	import ID	예
<code>aws_s3_bucket</code>	버킷 이름	<code>boaz26-w6-kdh1834-manual</code>
<code>aws_s3_bucket_versioning</code>	버킷 이름	<code>boaz26-w6-kdh1834-manual</code>
<code>aws_s3_bucket_public_access_block</code>	버킷 이름	<code>boaz26-w6-kdh1834-manual</code>
<code>aws_instance</code>	인스턴스 ID	<code>i-0abc123...</code>
<code>aws_security_group</code>	시큐리티 그룹 ID	<code>sg-0abc123...</code>
<code>aws_vpc</code>	VPC ID	<code>vpc-0abc123...</code>
<code>aws_iam_role</code>	역할 이름	<code>my-role</code>
<code>aws_route_table_association</code>	서브넷ID/라우트테이블ID	<code>subnet-aaa/rtb-bbb</code>

마지막 줄처럼 두 값을 슬래시나 쉼표로 이어 붙이는 것도 있습니다. 형식을 틀리면 대개 "그런 것이 없다"는 오류가 납니다.

이미 state 에 있는 주소로 다시 import 하면 `Resource already managed by Terraform` 으로 막힙니다. 이 메시지는 해결책까지 알려줍니다. 그 주소를 state 에서 먼저 빼라는 것이고, 그 명령이 `terraform state rm` 입니다. `state rm` 은 state 에서만 지우고 실물은 그대로 둡니다. 그래서 잘못 import 한 것을 되돌릴 때 씁니다.

💡 팁

import 를 하기 전에 `terraform state list` 를 한 번 보는 습관을 들이세요. 이미 있는지 없는지 3초면 확인됩니다. 실습워크북 B-2에서 그렇게 합니다.

Part 5. 고전 import 의 한계

여기가 이번 주에 겪어야 할 지점입니다. 7주차가 왜 필요한지가 여기서 나옵니다.

[개념] 13. No changes 가 "다 담았다"는 뜻은 아닙니다

버킷을 import 하고 나면 `plan` 이 `No changes` 라고 말합니다. 여기서 다 됐다고 생각하기 쉽습니다.

그런데 콘솔에서 그 버킷을 만들 때 버전 관리를 켜두었습니다. 코드를 다시 보세요.

```
resource "aws_s3_bucket" "manual" {  
  bucket = "boaz26-w6-kdh1834-manual"  
}
```

버전 관리에 대한 언급이 한 글자도 없습니다. 그런데도 `plan` 은 아무 말이 없습니다. Terraform 이 그 설정을 **비교 대상으로 삼고 있지 않기** 때문입니다.

`plan` 이 비교하는 것은 코드에 적힌 것과 실물의 대응하는 부분입니다. 코드에 없는 설정은 비교 대상이 아니고, 비교 대상이 아니면 차이도 없습니다. `No changes` 는 "코드에 적힌 범위 안에서는 어긋난 것이 없습니다"라는 뜻이지 "코드가 실물을 다 설명하고 있습니다"라는 뜻이 아닙니다.

이 상태에서 다음 사람이 이 코드만 읽으면 버전 관리가 켜져 있다는 것을 알 수 없습니다. 다른 리전에 같은 코드로 버킷을 만들면 그쪽은 버전 관리가 꺼진 채 만들어집니다. 코드가 인프라를 설명한다는 전제가 깨집니다.

⚠ 주의

모든 속성이 이렇게 아무 변경도 표시하지 않는 것은 아닙니다. 같은 "코드에 안 적은 것"이라도 provider 스키마에 따라 결과가 갈립니다. `tags` 는 코드에 없으면 `plan` 이 "실물의 태그를 지우겠다"고 나오고, 버전 관리는 코드에 없으면 아무 말도 하지 않습니다. 실습워크북 B-1에서 콘솔 버킷에 태그를 붙이지 말라고 하는 이유가 이것입니다.

어느 속성이 어느 쪽인지는 문서를 읽어서는 구분되지 않습니다. 결국 `plan` 을 돌려보고 확인하는 수밖에 없습니다. 이것도 고전 import 의 불편에 들어갑니다.

[개념] 14. S3 는 하나의 단위가 아닙니다

버전 관리를 코드로 옮기려면 무엇을 작성해야 할까요. `aws_s3_bucket` 블록 안에 `versioning` 이라고 작성할 것 같지만 그렇지 않습니다. AWS provider 4.x 부터 S3 버킷의 설정들이 **별도 리소스로 분리**되었습니다.

설정	리소스 이름
버전 관리	<code>aws_s3_bucket_versioning</code>
퍼블릭 접근 차단	<code>aws_s3_bucket_public_access_block</code>
기본 암호화	<code>aws_s3_bucket_server_side_encryption_configuration</code>
수명 주기 규칙	<code>aws_s3_bucket_lifecycle_configuration</code>
버킷 정책	<code>aws_s3_bucket_policy</code>
로깅	<code>aws_s3_bucket_logging</code>

콘솔에서는 버킷 하나를 생성한다고 생각하는데, Terraform 에서는 여러 리소스가 됩니다. 그리고 리소스마다 따로 import 해야 합니다.

그래서 실습 순서가 이렇게 됩니다.

1. 코드에 `aws_s3_bucket_versioning` 블록을 작성합니다
↓
2. `plan` 은 "이 리소스를 새로 만들겠다"고 합니다
실물에는 이미 활성화되어 있는데도 그렇습니다.
state 에 없기 때문입니다. 10번과 같은 상황입니다.
↓
3. 여기서 `apply` 하지 말고 `import` 합니다
↓
4. `plan` 이 변화를 감지하지 않습니다
이제 버전 관리가 코드에도 state 에도 실물에도 있습니다.

퍼블릭 접근 차단도 같은 방식으로 한 번 더 합니다. 리소스가 두 개면 이 과정을 두 번, 열 개면 열 번 합니다. 그리고 각 블록의 속성값은 `terraform state show` 로 실물을 확인하고 수동으로 입력해야 합니다.

! 꼭 기억

3번에서 `apply` 를 하지 말라고 한 이유는 두 가지입니다. 하나는 학습 순서입니다. `apply` 로 넘어가면 "이미 있는 것을 편입한다"는 오늘의 주제를 건너뛰게 됩니다. 다른 하나는 안전입니다. S3 의 버전 관리처럼 이미 켜져 있는 것을 다시 켜는 요청은 결과가 같지만, 리소스 종류에 따라서는 기존 설정을 덮어써서 되돌릴 수 없는 것도 있습니다. **실물이 이미 있으면 `apply` 가 아니라 `import` 입니다.**

이 반복이 오늘 겪어야 할 것입니다. 리소스 10개짜리 인프라를 코드로 옮긴다고 생각해 보세요. 주소를 적고, ID 를 찾고, `import` 하고, `state show` 로 열어보고, 속성을 직접 옮겨 적고, `plan` 이 `No changes` 가 될 때까지 반복합니다. 7주차의 `import` 블록과 `-generate-config-out` 이 이 과정 중 어디를 줄여주는지가 다음 주의 주제입니다.

[함정] 15. 모듈 안 리소스의 import 주소

week5에서 리소스 주소 앞에 `module.` 접두사가 붙는 것을 봤습니다. `import` 할 때도 그 주소를 그대로 적습니다.

`module.network.aws_vpc.this` 처럼 접두사까지 붙인 전체 주소여야 합니다.

주소를 작은따옴표로 감싸는 것을 잊지 마세요. week4·week5에서 `state show` 를 쓸 때와 같은 이유입니다. `for_each` 로 만든 리소스의 주소에는 대괄호와 큰따옴표가 들어 있고, 감싸지 않으면 셸이 그것을 먼저 해석해서 `zsh: no matches found` 가 납니다.

그리고 `for_each` 로 만든 리소스는 **인스턴스 하나하나를 따로 import 해야 합니다.** 서브넷이 두 개면 두 번입니다. 맵의 key 를 코드와 똑같이 적어야 하고, key 를 틀리면 다른 자리에 들어갑니다.

주소의 실제 모양은 실습워크북 B-7의 심화에 있습니다.

오늘 실습은 모듈을 쓰지 않습니다. 접두사가 붙으면 주소가 길어져서 `import` 자체에 집중하기 어렵기 때문입니다. 모듈 안 리소스 `import` 는 심화 과제(L3-★)에 있습니다.

Part 6. 비용과 정리

[개념] 16. 이번 주는 요금이 거의 없습니다. 그래도 destroy 는 합니다

오늘 만드는 것은 빈 S3 버킷 두 개입니다. S3 는 저장한 용량과 요청 수로 요금이 매겨지므로, 객체를 하나도 넣지 않은 버킷은 요금이 붙지 않습니다. EC2 를 만들지 않는 주차라 시간당 요금도 없습니다.

항목	개수	한 달
빈 S3 버킷 (<code>-drift</code> · <code>-manual</code>)	2	사실상 \$0
week3의 state 저장소 (S3 + DynamoDB)	1벌	\$0.01 미만

지난 다섯 주 중 처음으로 "지우지 않으면 얼마"가 문제가 안 되는 주차입니다. 그런데도 `destroy` 를 반드시 합니다. 이유가 셋입니다.

첫째, S3 버킷 이름은 전 세계에서 유일합니다. 남겨두면 그 이름을 다른 누구도 쓸 수 없고, 본인도 다음에 같은 이름으로 실습할 수 없습니다.

둘째, 다음 주차의 실습이 깨끗한 상태에서 시작합니다.

셋째, 17번이 이번 주 마무리의 핵심입니다.

[개념] 17. import 한 리소스는 destroy 대상이 됩니다

`manual` 버킷은 내가 콘솔에서 직접 만든 것입니다. Terraform 이 만든 것이 아닙니다. 그런데 import 를 한 순간부터 Terraform 의 관리 대상이 되었습니다.

`terraform destroy` 를 하면 어떻게 될까요. 오늘 만든 것이 전부 지워집니다. 콘솔에서 직접 만든 버킷도 함께 사라집니다.

11번에서 본 "앞으로 Terraform 이 관리한다"는 안내가 이런 뜻이었습니다. import 는 "Terraform 이 이것도 지울 수 있게 되었다"는 선언이기도 합니다.

실무에서 이것이 사고가 되는 자리가 있습니다. 운영 중인 데이터베이스나 버킷을 코드로 관리하려고 import 했는데, 나중에 누군가 코드에서 그 블록을 지우고 `apply` 를 하면 Terraform 은 "관리 대상이던 것이 코드에서 사라졌으니 지워야 한다"고 판단합니다. import 는 편입이면서 동시에 삭제 권한을 넘기는 일입니다.

🚫 절대 금지

`destroy` 는 실습워크북 Block C 에서 반드시 실행하세요. 그리고 week3의 S3 버킷(`*-tfstate`)과 DynamoDB 테이블(`*-tflock`)은 지우지 않습니다. 7주차까지 씁니다. 반대로 하면 state 저장소가 먼저 사라져서 오늘 만든 것을 지울 수단이 없어집니다.

버킷 안에 객체를 넣어두면 `destroy` 가 `BucketNotEmpty` 로 실패합니다. 콘솔에서 파일을 올려보지 마세요. 이미 올렸다면 버킷을 비운 뒤에 다시 `destroy` 해야 하고, 비우는 명령은 실습워크북 C-1에 있습니다.

Part 7. 다음 주로 넘기는 문제

[개념] 18. 세 문장으로 줄이면

1. 코드와 state와 실물은 세 곳에 따로 있습니다. `plan` 은 셋을 비교해서 보여주고, `apply` 는 실물을 코드에 맞추고, `apply -refresh-only` 는 state 를 실물에 맞춥니다.
2. `import` 는 state 만 채웁니다. 코드는 사람이 씁니다. 그래서 `No changes` 가 될 때까지 `state show` 를 보고 직접 옮겨 적는 과정이 남습니다.
3. Terraform 은 코드에 적힌 것만 비교합니다. 코드에 없는 설정은 실물에 있어도 `plan` 에 나오지 않습니다. S3 의 부속 리소스가 그 예입니다.

그리고 오늘 겪은 불편이 다음 주의 주제를 그대로 만들어냅니다.

오늘 본 것	Week7에서 다루는 지점
<code>terraform import</code> 를 리소스 개수만큼 반복 실행했습니다	<code>import</code> 블록을 코드에 작성하면 <code>apply</code> 한 번으로 여러 개를 편입합니다
<code>state show</code> 를 보고 속성을 수동으로 입력했습니다	<code>terraform plan -generate-config-out=파일.tf</code> 가 HCL 초안을 자동으로 생성합니다
<code>import</code> 명령 실행은 코드에 기록되지 않습니다	<code>import</code> 블록은 코드라서 PR 리뷰를 받고 git 히스토리에 남습니다
명령을 본인 노트북에서 직접 실행했습니다	GitHub Actions 에서 <code>plan</code> 을 실행하여 PR 에 결과를 표시하는 방식으로 변경합니다
콘솔 변경을 사람이 눈으로 감지했습니다	정기적으로 <code>plan</code> 을 실행하여 드리프트를 자동으로 감지하는 방법을 배웁니다

참고 문서

- [Import 개요](#) : Part 4 전체
- [Import 사용법](#) : 11 · 12번
- [Manage Resource Drift 튜토리얼](#) : Part 1 · Part 3. 사전 예습 자료
- [Refresh-Only 모드](#) : 8번
- [aws_s3_bucket 리소스](#) : 문서 맨 아래 Import 절
- [aws_s3_bucket_versioning 리소스](#) : 14번
- [import 블록](#) : 18번. 7주차 예습
- week2 개념워크북 9 · 12번 : state 주소와 교체(`-/+`)
- week3 개념워크북 Part 0 : 원격 backend 와 잠금
- week5 개념워크북 12번 : `moved` 가 state 만 고친다는 것

- week5 개념워크북 17번 : 오늘 회수한 표의 원래 자리

이 문서는 개념만 다루므로 명령과 화면을 실지 않았습니다. 여기서 이름으로만 언급한 오류 네 종류(`resource address ... does not exist in the configuration` · `Resource already managed by Terraform` · `Reference to undeclared resource` · `Invalid value for variable`)는 `solution/` 코드를 Terraform 1.15.8 · `darwin_arm64` 에서 실제로 실행해 재현한 것이고, 전문은 실습워크북에 있습니다. AWS 자격증명 없이 재현되므로 과금이 없습니다.

세 리소스의 import ID 가 모두 버킷 이름이라는 것은 AWS provider 문서(`aws_s3_bucket` · `aws_s3_bucket_versioning` · `aws_s3_bucket_public_access_block`)의 Import 절에서 확인했습니다. `aws_s3_bucket` 을 `bucket` 한 줄만 적고 import 했을 때 버전 관리가 diff 로 잡히지 않는 것과 태그는 diff 로 잡히는 것은 provider 스키마를 직접 조회해 확인했습니다. 버킷 리소스의 `versioning` 블록은 `Optional` 이면서 `Computed` 라 코드에 없으면 state 값을 그대로 두고, `tags` 는 `Computed` 가 아니라서 코드에 없으면 비우려 합니다.

5번이 설명하는 `plan` 화면의 두 덩어리(`Note: Objects have changed outside of Terraform` 과 `has changed · will be updated in-place`), 8번의 `-refresh-only` 승인 문구, 11번의 import 성공 안내는 같은 버전 바이너리의 메시지 목록에서 원문을 대조했습니다. 다만 그 문구가 실제로 이 실습에서 뜨는지는 apply 로 확인하지 않았습니다. 전문과 검증 범위는 실습워크북 끝의 각주에 있습니다. 비용은 빈 S3 버킷에 요금이 붙지 않는다는 사실에 근거한 것이고, week3 저장소의 \$0.01 은 week3 개념워크북에서 조회한 값을 옮겼습니다. 원화 환산은 넣지 않았습니다.

다음 → 실습워크북 을 펴고 `practice/` 폴더로 이동하세요.